

SOFTWARE REQUIREMENTS SPECIFICATION

# DCS Data Factory Pipeline Studio

v2.0 · React frontend + Databricks-native agent backend with persistent memory · 3-tier deterministic regeneration · Unity Catalog-native storage

|                |                                             |
|----------------|---------------------------------------------|
| DOCUMENT       | Software Requirements Specification (SRS)   |
| TARGET VERSION | v2.0                                        |
| REPLACES       | v1.0 (HTML studio + Copilot Chat clipboard) |
| STATUS         | Draft for review                            |
| OWNER          | Data Platform team                          |

LAST UPDATED 2026-05-25

Internal - For team review only

# DCS Data Factory Pipeline Studio · v2.0

## Requirements Specification

|                |                                                                  |
|----------------|------------------------------------------------------------------|
| Document       | Software Requirements Specification (SRS)                        |
| Product        | DCS Data Factory Pipeline Studio                                 |
| Target version | v2.0 — React frontend + Databricks-native agent backend          |
| Replaces       | v1.0 (single-HTML browser tool + GitHub Copilot Chat invocation) |
| Status         | Draft for review                                                 |
| Owner          | Data Platform team                                               |
| Last updated   | 2026-05-25                                                       |

**Goal of v2.0:** *scale the spec-driven multi-agent pipeline framework from one user's laptop to 100+ teams authoring + onboarding + monitoring 1000+ pipelines, with persistent state, agent memory for deterministic regeneration, and native Databricks integration.*

### ► 1 • Executive summary

v1.0 ships today as a single-HTML browser tool ( [tools/data-factory-pipeline-studio.html](#) ) that writes spec files to a linked project folder and emits a Copilot Chat invocation to clipboard. It works, but does not scale:

- **No multi-user state** — every user's drafts live in their own browser localStorage
- **No agent memory** — re-running the same spec twice produces different outputs (Copilot Chat is stateless across sessions)
- **No backend control** — model choice, cost, latency all delegated to whatever Copilot Chat has access to
- **No audit trail** — who edited what, when, why is invisible
- **Manual clipboard hand-off** — paste-into-Copilot is the integration; high friction

v2.0 keeps **100% of the v1.0 IP** (5 phase agents + 1 orchestrator + 19 skills + 38-rule catalog + spec format + output-path contract) and rebuilds only the **delivery surface** as a React + FastAPI + Databricks-native stack.

## ► 2 · Glossary

| TERM                         | DEFINITION                                                                                                                                                                                                        |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Phase agent</b>           | One of: <code>data_ingest</code> , <code>data_forge</code> , <code>data_prep</code> , <code>data_doc</code> , <code>data_lineage</code>                                                                           |
| <b>Master orchestrator</b>   | <code>data_pipeline_master</code> — the agent that delegates to all 4 phase agents in dependency order                                                                                                            |
| <b>Skill</b>                 | A reusable Python module that an agent calls (one of 19: <code>bronze_load</code> , <code>profiler</code> , <code>extraction</code> , etc.) — backed by a <code>SKILL.md</code> + <code>references/</code> folder |
| <b>Rule</b>                  | One of 38 deterministic rules in <code>.github/rules/categories/*.yaml</code> ( <code>pii_masking</code> , <code>regex</code> , <code>length_between</code> , etc.)                                               |
| <b>Spec</b>                  | A user-authored <code>spec.md</code> describing one pipeline · the input contract for an agent run                                                                                                                |
| <b>Spec override</b>         | A user-edited version of a spec stored separately so form-driven regeneration doesn't clobber manual edits                                                                                                        |
| <b>Subdomain</b>             | In the ingestion app's terminology, one Bronze target table — one ingestion pipeline can declare N subdomains                                                                                                     |
| <b>Pipeline health score</b> | 0–100 derived from 5 binary inputs: alerts configured · runbook present · data setup complete · profiling fresh · no active alerts                                                                                |
| <b>UC</b>                    | Unity Catalog                                                                                                                                                                                                     |
| <b>Databricks App</b>        | Hosted FastAPI/Flask app inside a Databricks workspace — gets workspace-scoped auth + UC access                                                                                                                   |
| <b>VS</b>                    | Databricks Vector Search                                                                                                                                                                                          |

## ► 3 · Scope

### 3.1 In scope (v2.0)

- React-based authoring frontend (replaces single-HTML studio)
- FastAPI backend on Databricks Apps
- Custom agent runtime calling Databricks model-serving endpoints (Claude family)
- 3-tier agent memory (conversational · per-pipeline · org-wide patterns)
- Chatbot panel (right rail · streams agent output via SSE)
- Unity Catalog storage for all operational state + spec files + agent artifacts
- Vector Search indexes for context retrieval
- RBAC via UC grants
- Audit log
- Migration path from v1.0 local drafts → v2.0 backend state

### 3.2 Out of scope (deferred to v2.1+)

- Mobile app
- Non-Databricks deployment (the framework is Databricks-first)
- Generic "skill builder UI" (skill definitions stay as Markdown files for now)
- Multi-region replication
- Real-time collaborative editing (Google-Docs-style live cursors)
- BYO model endpoint outside Databricks model serving

### 3.3 Out of scope permanently

- PII masking (Fiserv Protector handles encryption at source — v1.0 already removed this from claims)
  - Standalone runner notebooks (already removed in v1.0 — `manifest.json` IS the orchestrator)
-

## ► 4 · Functional requirements

### 4.1 Frontend (React SPA)

| REQ ID    | REQUIREMENT                                                                                                                                                                  | PRIORITY |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-FE-001 | Single-page React app served as a static bundle from a Databricks App                                                                                                        | MUST     |
| FR-FE-002 | Routes: <code>/portfolio</code> <code>/ingest/:id</code> <code>/eda/:id</code> <code>/dataprep/:id</code> <code>/docs/:id</code> <code>/onboard</code> <code>/lineage</code> | MUST     |
| FR-FE-003 | Forms use <code>react-hook-form</code> + Zod schemas matching every field in v1.0's <code>tools/data-factory-pipeline-studio.html</code>                                     | MUST     |
| FR-FE-004 | Sticky stage toolbar pattern on every form page (from v1.0)                                                                                                                  | MUST     |
| FR-FE-005 | Collapsible sections per panel (from v1.0) — open/closed state persisted server-side                                                                                         | MUST     |
| FR-FE-006 | Spec preview pane with RAW / RENDERED toggle (from v1.0)                                                                                                                     | MUST     |
| FR-FE-007 | Editable spec preview · saves override to backend (replaces v1.0 <code>localStorage</code> <code>STATE._specOverrides</code> )                                               | MUST     |
| FR-FE-008 | Floating chatbot panel on right rail · resizable · streams agent output via SSE                                                                                              | MUST     |
| FR-FE-009 | Home Dashboard with live KPI tiles + health score table fed by backend                                                                                                       | MUST     |
| FR-FE-010 | Lineage modal with Mermaid diagram + column inspector                                                                                                                        | MUST     |
| FR-FE-011 | Review panel showing all output/ artifacts for a pipeline · syntax-highlighted JSON · rendered Markdown                                                                      | MUST     |
| FR-FE-012 | Run-All-Stages wizard · same 4-step flow as v1.0 · final step calls <code>POST /pipelines/{id}/runs?phases=ALL</code>                                                        | MUST     |
| FR-FE-013 | Onboard new pipeline-type form — persists to <code>custom_pipeline_types</code> table for org-wide reuse                                                                     | MUST     |
| FR-FE-014 | Per-team view filter (Ingestion / Pre-Purposing / etc.) backed by <code>teams</code> table                                                                                   | MUST     |
| FR-FE-015 | All forms support draft auto-save every 5s to <code>spec_versions</code> table                                                                                               | MUST     |
| FR-FE-016 | Dark theme matches v1.0 Fiserv-orange visual language                                                                                                                        | SHOULD   |

| REQ ID    | REQUIREMENT                                                                                         | PRIORITY |
|-----------|-----------------------------------------------------------------------------------------------------|----------|
| FR-FE-017 | Light theme                                                                                         | NICE     |
| FR-FE-018 | Keyboard shortcuts: <b>Ctrl+S</b> save · <b>Ctrl+Enter</b> generate · <b>Ctrl+K</b> command palette | SHOULD   |

## 4.2 Backend API (FastAPI on Databricks Apps)

| REQ ID    | REQUIREMENT                                                                                           | PRIORITY |
|-----------|-------------------------------------------------------------------------------------------------------|----------|
| FR-BE-001 | FastAPI 0.110+ on Python 3.12                                                                         | MUST     |
| FR-BE-002 | Hosted as a Databricks App (gets workspace-scoped UC token, auto-mounted Volumes)                     | MUST     |
| FR-BE-003 | Auth via OAuth tokens passed through Databricks SSO                                                   | MUST     |
| FR-BE-004 | RESTful endpoints (see § 7 for full surface)                                                          | MUST     |
| FR-BE-005 | SSE endpoint for streaming agent output during runs                                                   | MUST     |
| FR-BE-006 | Pydantic models for every payload — schema validation server-side                                     | MUST     |
| FR-BE-007 | All endpoints idempotent where possible (PUT for updates, POST for creates)                           | MUST     |
| FR-BE-008 | Per-team RBAC enforced via UC group membership lookup                                                 | MUST     |
| FR-BE-009 | Audit log row written for every state-changing action                                                 | MUST     |
| FR-BE-010 | OpenAPI / Swagger UI exposed at <b>/docs</b> (FastAPI default)                                        | SHOULD   |
| FR-BE-011 | Rate limit: 100 req/min per user · 1000 req/min per team                                              | SHOULD   |
| FR-BE-012 | Background jobs (health-score computation · profiling) run as Databricks Workflows triggered via REST | MUST     |

### 4.3 Agent runtime

| REQ ID    | REQUIREMENT                                                                                                                                                | PRIORITY |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-AR-001 | Python orchestrator that loads agent prompts from <code>/Volumes/df_studio/agents/*.md</code> at process start                                             | MUST     |
| FR-AR-002 | LangGraph (or hand-rolled equivalent) tool-use loop                                                                                                        | MUST     |
| FR-AR-003 | Calls Databricks model-serving endpoint for the agent's declared <code>model</code> (e.g. <code>databricks-claude-3-7-sonnet</code> )                      | MUST     |
| FR-AR-004 | Honors <code>fallback_models</code> list from agent YAML frontmatter — automatic failover                                                                  | MUST     |
| FR-AR-005 | Each of the 19 skills wrapped as a Python tool the agent can call · tool name = skill ID (e.g. <code>ingestion.bronze_load</code> )                        | MUST     |
| FR-AR-006 | Streams token output back to API layer via internal async channel                                                                                          | MUST     |
| FR-AR-007 | Writes outputs to <code>/Volumes/df_studio/output/&lt;phase&gt;/&lt;pipeline&gt;/...</code> following v1.0 path contract                                   | MUST     |
| FR-AR-008 | Per-subdomain ingestion configs in <code>configs/&lt;subdomain_slug&gt;/</code> (preserves v1.0 layout)                                                    | MUST     |
| FR-AR-009 | Master orchestrator ( <code>data_pipeline_master</code> ) handles partial spec sets — phases without attached specs reported as <code>NOT_ATTEMPTED</code> | MUST     |
| FR-AR-010 | Every run produces a <code>runs</code> table row with status + token count + errors                                                                        | MUST     |
| FR-AR-011 | Failed runs do NOT block downstream phases when run via master · master stops at the failure boundary and reports                                          | MUST     |
| FR-AR-012 | Skill loading reads <code>SKILL.md</code> + <code>references/*</code> at runtime · no compile-time bundling                                                | MUST     |
| FR-AR-013 | Rule catalog loaded from <code>/Volumes/df_studio/rules/catalog.yaml</code> + categories · cached for 60s                                                  | MUST     |
| FR-AR-014 | Agent run timeout: 10 minutes for single phase · 30 minutes for master                                                                                     | SHOULD   |
| FR-AR-015 | Cancellable mid-run via <code>DELETE /pipelines/{id}/runs/{run_id}</code>                                                                                  | SHOULD   |

### 4.4 Storage

See § 6 for full storage layout. Summary:

| REQ ID    | REQUIREMENT                                                                                                                  | PRIORITY |
|-----------|------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-ST-001 | All operational state in Unity Catalog Delta tables under <code>df_studio.metadata.*</code>                                  | MUST     |
| FR-ST-002 | All agent artifacts (specs, outputs) in Unity Catalog Volume under <code>/Volumes/df_studio/</code>                          | MUST     |
| FR-ST-003 | Vector Search indexes for skill corpus, past specs, rule catalog, runbook history                                            | MUST     |
| FR-ST-004 | Every spec save creates a new row in <code>spec_versions</code> — never overwrites · audit-friendly                          | MUST     |
| FR-ST-005 | Spec overrides stored separately from generated specs so <code>buildSpec()</code> logic from v1.0 still applies server-side  | MUST     |
| FR-ST-006 | Delta retention policy: <code>spec_versions</code> = 1 year · <code>runs</code> = 90 days · <code>audit_log</code> = 7 years | SHOULD   |
| FR-ST-007 | UC grants used for per-team RBAC — no app-level permission logic                                                             | MUST     |

## 4.5 Agent memory

| REQ ID     | REQUIREMENT                                                                                                                                       | PRIORITY |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-MEM-001 | 3-tier memory model (see § 8)                                                                                                                     | MUST     |
| FR-MEM-002 | Tier 2 (pipeline-scoped, durable) — Delta table <code>agent_memory</code> with <code>(pipeline_id, agent, memory_key, memory_value)</code> schema | MUST     |
| FR-MEM-003 | When re-running an existing pipeline, agent loads ALL Tier-2 entries first and surfaces them in the prompt as "PRIOR DECISIONS"                   | MUST     |
| FR-MEM-004 | Tier-1 (chat session) memory ephemeral · stored in process or Redis · TTL = chat session lifetime                                                 | MUST     |
| FR-MEM-005 | Tier-3 (org-wide patterns) — Vector Search index over past pipelines · agent queries it before composing                                          | MUST     |
| FR-MEM-006 | Memory entries are user-visible + user-editable (a "Pipeline Memory" panel in the UI) — full transparency                                         | MUST     |
| FR-MEM-007 | Memory entries have schema validation (defined keys per agent) — no free-form key explosion                                                       | MUST     |
| FR-MEM-008 | Re-running an unchanged spec produces byte-identical output via memory replay                                                                     | MUST     |



## 4.6 Chatbot

| REQ ID    | REQUIREMENT                                                                                                                                          | PRIORITY |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-CB-001 | Right-rail panel · width = 360px default · user-resizable to 600px                                                                                   | MUST     |
| FR-CB-002 | Streaming responses via SSE                                                                                                                          | MUST     |
| FR-CB-003 | Pipeline-scoped — chat history persists per pipeline · separate thread per <code>pipeline_id</code>                                                  | MUST     |
| FR-CB-004 | Chat actions can populate form fields (one-way: chat → form, never form → chat)                                                                      | MUST     |
| FR-CB-005 | "Apply suggestion" button on chat replies — bulk-fills the form                                                                                      | MUST     |
| FR-CB-006 | Form remains the source of truth before Generate · chat is advisory                                                                                  | MUST     |
| FR-CB-007 | History stored in <code>chat_turns</code> table (pipeline_id, turn_id, role, content, created_at)                                                    | MUST     |
| FR-CB-008 | "/" slash commands: <code>/skill &lt;name&gt;</code> <code>/rule &lt;name&gt;</code> <code>/example &lt;topic&gt;</code> show contextual info inline | SHOULD   |
| FR-CB-009 | Markdown rendering in replies                                                                                                                        | MUST     |

## 4.7 Observability + dashboarding

| REQ ID     | REQUIREMENT                                                                                                                             | PRIORITY |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-OBS-001 | Home Dashboard with 5 KPI tiles: pipelines · active this week · spec coverage % · open gaps · PII columns                               | MUST     |
| FR-OBS-002 | Pipeline health table — one row per pipeline · sortable by health score · click to open                                                 | MUST     |
| FR-OBS-003 | Health score computed nightly via Databricks Workflow · 5 inputs (alerts ok · runbook ok · data setup ok · profiling fresh · no alerts) | MUST     |
| FR-OBS-004 | Activity feed — last 10 runs across the project                                                                                         | MUST     |
| FR-OBS-005 | Governance widget — spec coverage · PII masked · runbook present (as progress bars)                                                     | MUST     |
| FR-OBS-006 | Per-pipeline run history page · timeline view · click any run to see its artifacts                                                      | MUST     |
| FR-OBS-007 | Alert hooks — Slack + PagerDuty webhooks on health-score drops below 60 · drift detected · freshness violation                          | SHOULD   |
| FR-OBS-008 | Lakehouse Monitoring on the <code>health_scores</code> Delta — auto-detects anomalies                                                   | SHOULD   |

## 4.8 Onboarding new pipeline types

| REQ ID     | REQUIREMENT                                                                                                                                | PRIORITY |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-ONB-001 | Onboard form (replaces v1.0 Onboard tab) — name · icon · domain · subdomain · description · agent · phase · columns · spec template · team | MUST     |
| FR-ONB-002 | Saves to <code>custom_pipeline_types</code> Delta table — org-wide visibility                                                              | MUST     |
| FR-ONB-003 | Once onboarded, the new pipeline type appears as a new tab in any team's view (filtered by <code>team</code> column)                       | MUST     |
| FR-ONB-004 | Custom pipeline types can be shared/forked across teams                                                                                    | SHOULD   |
| FR-ONB-005 | Approval workflow for onboarding new types (optional · enabled per-org)                                                                    | NICE     |

## 4.9 Master orchestrator (Run All Stages)

| REQ ID     | REQUIREMENT                                                                                                  | PRIORITY |
|------------|--------------------------------------------------------------------------------------------------------------|----------|
| FR-RUN-001 | Wizard walks user through Ingest → EDA → Data Prep → Document specs · same as v1.0                           | MUST     |
| FR-RUN-002 | "Submit" calls <code>POST /pipelines/{id}/runs</code> with <code>phases=[ingestion,eda,dataprep,docs]</code> | MUST     |
| FR-RUN-003 | Backend invokes <code>data_pipeline_master</code> agent with all 4 attached specs                            | MUST     |
| FR-RUN-004 | Partial spec sets (skipped phases) reported as <code>NOT_ATTEMPTED</code> in the end-of-run summary          | MUST     |
| FR-RUN-005 | Live streaming view shows phase-by-phase progress in the React UI · no clipboard / VS Code launch            | MUST     |
| FR-RUN-006 | Failure boundary — if phase 2 fails, phases 3 + 4 marked <code>BLOCKED</code> not <code>NOT_ATTEMPTED</code> | MUST     |

## 4.10 Integrations

| REQ ID     | REQUIREMENT                                                                                                          | PRIORITY |
|------------|----------------------------------------------------------------------------------------------------------------------|----------|
| FR-INT-001 | KNL Validation — pipe <code>data_prep</code> conversion output through KNL · results into <code>manifest.json</code> | SHOULD   |
| FR-INT-002 | Confluence Runbook publish — auto-push <code>runbook.md</code> to a Confluence space · diff-aware updates            | SHOULD   |
| FR-INT-003 | GitHub PR opener — generate PR with spec + outputs to a pipelines repo                                               | NICE     |
| FR-INT-004 | Slack notification on Run-All complete                                                                               | SHOULD   |

## ► 5 · Non-functional requirements

### 5.1 Performance

- **NFR-PERF-001:** Frontend page load < 2s on Chrome 120+ over a typical corporate VPN
- **NFR-PERF-002:** API p95 latency < 300ms for read endpoints · < 1s for spec writes
- **NFR-PERF-003:** Agent run streaming: first token < 3s · sustained > 30 tokens/sec
- **NFR-PERF-004:** Dashboard load < 1s for projects with up to 500 pipelines

### 5.2 Scale

- **NFR-SCALE-001:** Support 100+ concurrent users on a single Databricks App instance

- **NFR-SCALE-002:** Support 10,000+ pipelines across all teams in one workspace
- **NFR-SCALE-003:** Vector Search indexes scale to 100,000+ embedded documents
- **NFR-SCALE-004:** Audit log retention 7 years · partition by year · monthly compaction

### 5.3 Security + compliance

- **NFR-SEC-001:** All auth via Databricks SSO (OAuth tokens · no app-level password store)
- **NFR-SEC-002:** All API endpoints require valid workspace token
- **NFR-SEC-003:** UC grants enforce read/write boundaries between teams
- **NFR-SEC-004:** No PII in spec.md files · Fiserv Protector encrypts at source · framework treats source data as already-tokenized
- **NFR-SEC-005:** No secrets in `_meta.unresolved` or in agent prompt · use Databricks secret scopes only
- **NFR-SEC-006:** Audit log immutable (Delta CHANGE\_DATA\_FEED enabled · no UPDATE/DELETE on existing rows)

### 5.4 Availability

- **NFR-AVAIL-001:** Target 99.5% uptime within Databricks workspace SLA
- **NFR-AVAIL-002:** Graceful degradation when model-serving endpoint is down · queue runs for retry · UI shows "model unavailable, queued" banner
- **NFR-AVAIL-003:** Read-only mode available when write path fails (users can still view existing pipelines + outputs)

### 5.5 Browser support

- **NFR-COMPAT-001:** Chrome / Edge 120+ (primary)
- **NFR-COMPAT-002:** Firefox 120+ (best-effort)
- **NFR-COMPAT-003:** Safari 17+ (best-effort · Mac-only)
- **NFR-COMPAT-004:** No IE11 · no Chrome < 100

## ► 6 · Storage layout (detailed)

### 6.1 Unity Catalog schema

```
CREATE CATALOG IF NOT EXISTS df_studio;
CREATE SCHEMA IF NOT EXISTS df_studio.metadata;
CREATE SCHEMA IF NOT EXISTS df_studio.vs;           -- vector search indexes
CREATE VOLUME IF NOT EXISTS df_studio.metadata.files; -- managed Volume
```

## 6.2 Tables (Delta · partitioned where noted)

```
-- 6.2.1 · Pipelines master
CREATE TABLE df_studio.metadata.pipelines (
  pipeline_id  STRING NOT NULL,      -- ulid
  name         STRING NOT NULL,
  phase        STRING NOT NULL,      -- ingestion | eda | dataprep | docs | <custom>
  owner_email  STRING NOT NULL,
  team         STRING NOT NULL,
  domain       STRING,               -- banking, healthcare, etc.
  subdomain    STRING,
  status       STRING,               -- draft | active | archived
  created_at   TIMESTAMP NOT NULL,
  updated_at   TIMESTAMP NOT NULL,
  current_spec_version INT NOT NULL,
  health_score INT,                  -- 0-100 · refreshed by nightly Workflow
  health_score_at TIMESTAMP
) USING DELTA
  TBLPROPERTIES (delta.enableChangeDataFeed = true);

-- 6.2.2 · Spec versions (audit trail · every save = new row)
CREATE TABLE df_studio.metadata.spec_versions (
  pipeline_id  STRING NOT NULL,
  version      INT NOT NULL,
  spec_md      STRING NOT NULL,      -- full markdown
  generated_by  STRING NOT NULL,      -- 'form' | 'override' | 'chat' | 'master_run'
  generated_at  TIMESTAMP NOT NULL,
  diff_summary  STRING,               -- "added 2 subdomains, removed PII column"
  user_email   STRING NOT NULL,
  PRIMARY KEY (pipeline_id, version)
) USING DELTA
  PARTITIONED BY (DATE_TRUNC('month', generated_at));

-- 6.2.3 · Spec overrides (per-user hand-edits · replaces LocalStorage)
CREATE TABLE df_studio.metadata.spec_overrides (
  pipeline_id  STRING NOT NULL,
  user_email   STRING NOT NULL,
  override_md  STRING NOT NULL,
  applied_at   TIMESTAMP NOT NULL,
  reason       STRING,
  PRIMARY KEY (pipeline_id, user_email)
) USING DELTA;

-- 6.2.4 · Agent runs
CREATE TABLE df_studio.metadata.runs (
  run_id       STRING NOT NULL,      -- ulid
  pipeline_id  STRING NOT NULL,
  agent        STRING NOT NULL,      -- data_ingest, data_forge, ..., data_pipeline_master
  started_at   TIMESTAMP NOT NULL,
  finished_at  TIMESTAMP,
  status       STRING NOT NULL,      -- queued | running | passed | failed | cancelled
  model        STRING,               -- databricks-claude-3-7-sonnet
  fallback_used BOOLEAN,
  token_count_in BIGINT,
  token_count_out BIGINT,
  output_path  STRING,               -- Volume path

```

```

errors_json    STRING,
user_email     STRING NOT NULL,
PRIMARY KEY (run_id)
) USING DELTA
PARTITIONED BY (DATE_TRUNC('day', started_at));

-- 6.2.5 · Agent memory (TIER 2 · deterministic-output engine)
CREATE TABLE df_studio.metadata.agent_memory (
  pipeline_id   STRING NOT NULL,
  agent         STRING NOT NULL,
  memory_key    STRING NOT NULL,          -- 'chosen_load_query_mode', 'inferred_pk', etc.
  memory_value  STRING NOT NULL,          -- JSON-encoded value
  created_at    TIMESTAMP NOT NULL,
  updated_at    TIMESTAMP NOT NULL,
  user_visible  BOOLEAN DEFAULT true,     -- false for internal-only state
  PRIMARY KEY (pipeline_id, agent, memory_key)
) USING DELTA;

-- 6.2.6 · Health scores (drives the dashboard)
CREATE TABLE df_studio.metadata.health_scores (
  pipeline_id   STRING NOT NULL,
  computed_at   TIMESTAMP NOT NULL,
  alerts_ok     BOOLEAN NOT NULL,
  runbook_ok    BOOLEAN NOT NULL,
  data_setup_ok BOOLEAN NOT NULL,
  profiling_ok  BOOLEAN NOT NULL,
  drift_ok      BOOLEAN NOT NULL,
  score         INT NOT NULL,             -- 0-100
  PRIMARY KEY (pipeline_id, computed_at)
) USING DELTA
PARTITIONED BY (DATE_TRUNC('month', computed_at));

-- 6.2.7 · Custom pipeline types (replaces Onboard tab's localStorage)
CREATE TABLE df_studio.metadata.custom_pipeline_types (
  type_id       STRING NOT NULL,          -- slug
  name          STRING NOT NULL,
  icon          STRING,
  domain        STRING,
  subdomain     STRING,
  description    STRING,
  agent         STRING,                   -- which agent.md to attach
  phase         STRING,
  columns       STRING,                   -- JSON: default columns/schema
  spec_template STRING,                   -- markdown template
  team          STRING NOT NULL,
  created_by    STRING NOT NULL,
  created_at    TIMESTAMP NOT NULL,
  PRIMARY KEY (type_id)
) USING DELTA;

-- 6.2.8 · Chat turns
CREATE TABLE df_studio.metadata.chat_turns (
  pipeline_id   STRING NOT NULL,
  turn_id       STRING NOT NULL,
  role          STRING NOT NULL,          -- user | assistant | tool
  content       STRING NOT NULL,

```

```

    created_at      TIMESTAMP NOT NULL,
    user_email      STRING NOT NULL,
    PRIMARY KEY (pipeline_id, turn_id)
) USING DELTA
    PARTITIONED BY (DATE_TRUNC('month', created_at));

-- 6.2.9 · Audit Log (immutable · 7-year retention)
CREATE TABLE df_studio.metadata.audit_log (
    event_id        STRING NOT NULL,
    user_email      STRING NOT NULL,
    pipeline_id     STRING,
    action          STRING NOT NULL,      -- spec.save, run.create, override.apply, type.onboard, ...
    payload_json    STRING,
    created_at      TIMESTAMP NOT NULL,
    PRIMARY KEY (event_id)
) USING DELTA
    PARTITIONED BY (DATE_TRUNC('month', created_at))
    TBLPROPERTIES (delta.enableChangeDataFeed = false);

-- 6.2.10 · Teams + members (RBAC via UC groups · denormalised for fast lookup)
CREATE TABLE df_studio.metadata.teams (
    team_id         STRING NOT NULL PRIMARY KEY,
    name            STRING NOT NULL,
    uc_group        STRING NOT NULL,      -- the UC group that grants access
    default_phases  STRING                -- JSON list of phase ids
) USING DELTA;

```

## 6.3 Unity Catalog Volume tree

```

/Volumes/df_studio/metadata/files/
├─ agents/                                     ← 6 prompts · single source of truth
│  ├─ data_ingest.md
│  ├─ data_forge.md
│  ├─ data_prep.md
│  ├─ data_doc.md
│  ├─ data_lineage.md
│  └─ data_pipeline_master.md
├─ skills/                                    ← 19 SKILL.md + references/
│  ├─ ingestion/
│  │  ├─ bronze_load/SKILL.md
│  │  ├─ bronze_load/references/example.json
│  │  ├─ profiler/...
│  │  └─ pipeline/...
│  ├─ eda/...
│  ├─ dataprep/...
│  ├─ docs/pipeline_doc/...
│  └─ lineage/builder/...
├─ rules/                                    ← 38-rule catalog
│  ├─ catalog.yaml
│  ├─ rule_schema.yaml
│  └─ categories/*.yaml
├─ specs/                                    ← latest pointer to spec_versions content
│  └─ <phase>/<pipeline>/spec.md
├─ output/                                   ← agent-produced artifacts (v1.0 layout preserved)
│  ├─ ingestion/<pipeline>/configs/<subdomain>/{bronze_load,profiler}.json
│  ├─ ingestion/<pipeline>/configs/manifest.json
│  ├─ dataprep/<pipeline>/configs/{6-stage}.json
│  ├─ eda/<pipeline>/{notebooks,reports,sql}/
│  ├─ docs/<pipeline>/{ingestion_info,dataprep_info,runbook}.md
│  └─ lineage/<scope>/{lineage,column_index,impact_*}.md
└─ query_libraries/                          ← user-uploaded .sql for query_path mode
   └─ <team>/<subdomain>/extract.sql

```



## 6.4 Vector Search indexes

| INDEX                                     | SOURCE TABLE / VOLUME                                                                      | EMBEDDING MODEL                      | USE CASE                                                                             |
|-------------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------|--------------------------------------------------------------------------------------|
| <code>df_studio.vs.skill_corpus</code>    | Indexes every <code>SKILL.md</code> + <code>references/*.md</code> content from the Volume | <code>databricks-gte-large-en</code> | Agent finds the 3 most relevant skills for "I want to ingest a CSV with PII columns" |
| <code>df_studio.vs.past_specs</code>      | Every row in <code>spec_versions</code> (chunked by section)                               | same                                 | "Find pipelines similar to this one I'm describing"                                  |
| <code>df_studio.vs.rule_catalog</code>    | Each rule from <code>categories/*.yaml</code> (id + description + args)                    | same                                 | Agent maps column tags → applicable rules without scanning all 38                    |
| <code>df_studio.vs.runbook_history</code> | Past <code>runbook.md</code> content                                                       | same                                 | New runbook drafts pull patterns from past on-call learnings                         |

## ► 7 · API surface

### 7.1 Pipelines

```

GET    /api/v1/pipelines      list (filter by team, phase, status)
POST   /api/v1/pipelines      create
GET    /api/v1/pipelines/{id}  read (spec + override applied)
PUT    /api/v1/pipelines/{id}  update metadata
DELETE /api/v1/pipelines/{id}  soft delete (status=archived)

```

### 7.2 Specs

```

GET    /api/v1/pipelines/{id}/spec      current spec (with override merged)
PUT    /api/v1/pipelines/{id}/spec      save new version
GET    /api/v1/pipelines/{id}/spec/versions  list versions
GET    /api/v1/pipelines/{id}/spec/versions/{v}  read specific version
GET    /api/v1/pipelines/{id}/override      read user's override (if any)
PUT    /api/v1/pipelines/{id}/override      save user's override
DELETE /api/v1/pipelines/{id}/override      reset to form-generated

```

### 7.3 Runs (agent invocations)

```

POST   /api/v1/pipelines/{id}/runs      kick off · body: {phases: [ingestion,eda,...]}
GET    /api/v1/pipelines/{id}/runs      list runs for this pipeline
GET    /api/v1/pipelines/{id}/runs/{run_id}  read run details + artifact paths
GET    /api/v1/pipelines/{id}/runs/{run_id}/stream  SSE token stream
DELETE /api/v1/pipelines/{id}/runs/{run_id}  cancel

```

## 7.4 Chat

|      |                                    |                                              |
|------|------------------------------------|----------------------------------------------|
| POST | /api/v1/chat/{pipeline_id}/turn    | send user message · streams response via SSE |
| GET  | /api/v1/chat/{pipeline_id}/history | load chat history                            |

## 7.5 Catalog + dashboard

|     |                                |                                          |
|-----|--------------------------------|------------------------------------------|
| GET | /api/v1/catalog/agents         | list of 6 agents + their metadata        |
| GET | /api/v1/catalog/skills         | list of 19 skills (filterable by phase)  |
| GET | /api/v1/catalog/rules          | list of 38 rules                         |
| GET | /api/v1/catalog/pipeline-types | list of custom + built-in pipeline types |
| GET | /api/v1/dashboard/kpis         | 5 KPI tiles                              |
| GET | /api/v1/dashboard/health       | health scores (sortable / paginated)     |
| GET | /api/v1/dashboard/activity     | last N runs                              |
| GET | /api/v1/dashboard/governance   | coverage + compliance progress bars      |

## 7.6 Onboard

|        |                                     |                                   |
|--------|-------------------------------------|-----------------------------------|
| POST   | /api/v1/onboard/pipeline-types      | create new pipeline type          |
| GET    | /api/v1/onboard/pipeline-types/{id} | read                              |
| DELETE | /api/v1/onboard/pipeline-types/{id} | remove (only by creator or admin) |

## 7.7 Lineage

|      |                                        |                                |
|------|----------------------------------------|--------------------------------|
| GET  | /api/v1/lineage?scope=project          | whole-project DAG              |
| GET  | /api/v1/lineage?scope=pipeline&id={id} | per-pipeline DAG               |
| GET  | /api/v1/lineage/columns/{column_name}  | column-impact analysis         |
| POST | /api/v1/lineage/generate               | trigger data_lineage agent run |

## 7.8 Memory (Tier 2 · per-pipeline)

|        |                                     |                         |
|--------|-------------------------------------|-------------------------|
| GET    | /api/v1/pipelines/{id}/memory       | list all memory entries |
| PUT    | /api/v1/pipelines/{id}/memory/{key} | set value               |
| DELETE | /api/v1/pipelines/{id}/memory/{key} | forget                  |

# ► 8 · Agent memory schema

## 8.1 Tier 1 · Conversational (ephemeral)

- Stored in Redis or in-process dict
- TTL: chat session lifetime
- Holds: current chat turn, last 10 turns, tool-call intermediate state
- NOT durable across page reload

## 8.2 Tier 2 · Pipeline-scoped (durable · the determinism engine)

- Stored in `df_studio.metadata.agent_memory` Delta table
- Keyed by `(pipeline_id, agent, memory_key)`
- Loaded into the agent prompt as a `## Prior decisions` section before any new run

**Standard keys** (per agent · defined in `agents/<agent>.md` YAML frontmatter under `memory_keys`):

| AGENT                    | MEMORY KEY                              | EXAMPLE VALUE                                            |
|--------------------------|-----------------------------------------|----------------------------------------------------------|
| <code>data_ingest</code> | <code>inferred_pk</code>                | <code>customer_id</code>                                 |
| <code>data_ingest</code> | <code>chosen_load_query_mode</code>     | <code>inline / file / table</code>                       |
| <code>data_ingest</code> | <code>source_db_schema</code>           | <code>BANK.DBO</code>                                    |
| <code>data_ingest</code> | <code>watermark_column</code>           | <code>updated_at</code>                                  |
| <code>data_forge</code>  | <code>chosen_entity_alias</code>        | <code>consumer_header</code>                             |
| <code>data_forge</code>  | <code>relationship_hints_applied</code> | <code>["customer_id-&gt;address", ...]</code>            |
| <code>data_prep</code>   | <code>source_mode</code>                | <code>header_query / table / extract_query</code>        |
| <code>data_prep</code>   | <code>pii_masking_strategy</code>       | <code>{"ssn": "sha2_256", "dob": "truncate_date"}</code> |
| <code>data_doc</code>    | <code>mode</code>                       | <code>ingest / dataprep / runbook</code>                 |
| <code>data_doc</code>    | <code>confluence_page_id</code>         | <code>12345678</code> (after first publish)              |

## 8.3 Tier 3 · Org-wide patterns (learned across pipelines)

- Stored in `df_studio.vs.past_specs` + a small `pipeline_patterns` Delta
- Vector search retrieves "pipelines like this" before agent composes
- Agent receives top 3 matches as background context (not as enforcement)
- User can disable per-pipeline via a "use patterns" toggle

## ► 9 · Migration plan (6 phases · ~3-4 months)

### Phase 0 · Foundation (1-2 weeks)

- Create `df_studio` catalog · schemas · all 10 Delta tables (DDL above)
- Create `df_studio.metadata.files` Volume
- Bulk-upload current `agents/` `skills/` `rules/` from v1.0 repo into the Volume
- Write `loaders.py` — reads YAML frontmatter from agents · loads skill modules · validates rule catalog
- **Exit criterion:** `python -c "from loaders import *; print(list_all_agents())"` returns the 6 agents from UC Volume

### Phase 1 · Backend API (3-4 weeks)

- FastAPI scaffolded on a Databricks App

- Implement § 7.1 + 7.2 + 7.5 (read-only catalog endpoints + spec CRUD)
- Point the existing v1.0 HTML studio at the new API (just for spec save/load) — proves the backend works while UI is untouched
- **Exit criterion:** v1.0 HTML studio runs against the API · specs persist in `spec_versions`

## Phase 2 · Agent runtime (3-4 weeks)

- Implement LangGraph orchestrator
- Wrap 19 skills as Python tools
- Wire to Databricks model serving ( `databricks-claude-3-7-sonnet` + fallbacks)
- SSE streaming endpoint
- `POST /runs` endpoint
- **Exit criterion:** kick off a `data_ingest` run from the API · output lands in `/Volumes/df_studio/output/ingestion/<test>/configs/...`

## Phase 3 · React frontend (4-6 weeks)

- Scaffold Vite + React + TanStack Router
- Port all 6 tabs from v1.0 (Portfolio · Ingestion · EDA · Data Prep · Document · Onboard)
- Sticky toolbar + collapsible sections + spec preview + edit override
- Run-All Stages wizard
- Real-time SSE stream view
- **Exit criterion:** a new pipeline can be authored entirely in the React UI · spec saves to backend · run completes · outputs visible in Review panel

## Phase 4 · Memory + Vector Search (2-3 weeks)

- Tier 2 `agent_memory` table wired into agent runtime
- Vector Search indexes set up (skill corpus · past specs · rule catalog · runbook history)
- "Regenerate" button that replays memory for deterministic output
- Memory panel in UI showing prior decisions per pipeline
- **Exit criterion:** regenerating an unchanged spec produces byte-identical output 3 times in a row

## Phase 5 · Chatbot + Observability (3-4 weeks)

- Right-rail chatbot panel with SSE streaming
- Chat → form one-way population
- Home Dashboard (KPIs · health table · activity · governance · quick lineage)
- Nightly health-score Workflow
- **Exit criterion:** dashboard populated · chatbot can fill 80% of a typical ingestion spec from a 3-turn conversation

## Phase 6 · Roadmap items (4-6 weeks)

- KNL Validation Integration
- Confluence Runbook publish
- Alerts & Monitoring (Slack · PagerDuty)
- AppOps Handover Package

- **Exit criterion:** roadmap section on architecture HTML moves to  Done

## ► 10 · Acceptance criteria (how we know v2 is done)

| #     | CRITERION                                                                                                       | OWNER      |
|-------|-----------------------------------------------------------------------------------------------------------------|------------|
| AC-01 | All 657 v1.0 tests pass on the v2 codebase (logic ports to backend)                                             | Eng        |
| AC-02 | A new user can author a 3-subdomain ingestion pipeline + run it end-to-end in < 10 minutes without reading docs | UX         |
| AC-03 | Same spec re-run twice produces byte-identical output (Tier-2 memory works)                                     | Eng        |
| AC-04 | 100 concurrent users on the Databricks App without p95 latency exceeding 1s                                     | SRE        |
| AC-05 | Health score updates within 1h of a pipeline change                                                             | Eng        |
| AC-06 | Audit log captures every state-changing action with user + timestamp                                            | Compliance |
| AC-07 | RBAC: Team A cannot see Team B's pipelines unless explicitly granted                                            | Sec        |
| AC-08 | v1.0 specs/outputs can be batch-imported (one script run) without data loss                                     | Eng        |
| AC-09 | All 5 phase agents + master orchestrator pass their "happy path" test in CI                                     | Eng        |
| AC-10 | Architecture HTML (v2 version) renders + reflects the deployed state                                            | Docs       |

## ► 11 · Risks + mitigations

| RISK                                                                       | LIKELIHOOD | IMPACT | MITIGATION                                                                                                                                          |
|----------------------------------------------------------------------------|------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Databricks model-serving Claude endpoint rate limits                       | Medium     | High   | Fallback chain in agent frontmatter · queue + retry · UI shows "queued" banner                                                                      |
| Tier-2 memory grows unboundedly                                            | Low        | Medium | Schema-validated keys (no free-form keys) · max 50 entries per pipeline · TTL 1 year                                                                |
| React frontend becomes too complex (current studio is ~7800 lines of HTML) | Medium     | Medium | Component-driven · shadcn/ui · TanStack Router for clean route boundaries · Storybook for component review                                          |
| Vector Search costs balloon                                                | Low        | Medium | Index only when content changes (not nightly) · monitor token usage                                                                                 |
| Migrating v1.0 specs hits edge cases (malformed markdown)                  | Medium     | Low    | Migration script has a <code>--dry-run</code> mode that reports diff vs. canonical · manual review for outliers                                     |
| Team A's spec template conflicts with Team B's                             | Low        | Medium | <code>custom_pipeline_types</code> keyed by team — explicit fork/share workflow                                                                     |
| Agent prompt drift between v1.0 and v2.0                                   | Medium     | High   | Single source of truth = Volume files · <code>loaders.py</code> is the only path · CI test that loads + diffs against committed agents/ in the repo |

## ► 12 · Open questions

1. **Branding:** keep "DCS Data Factory Pipeline Studio" or rename for v2? — TBD by product
2. **Streaming UI for master orchestrator:** phase-by-phase progress bar OR raw stream of agent output? — recommend phase progress bar with collapsible raw stream
3. **Vector Search vs. simple metadata filter for skill discovery:** VS is more flexible but adds latency. For 19 skills, do we even need VS or is a metadata filter on the catalog table enough? — start without VS for skills, add if discovery quality degrades
4. **GitHub PR opener** (FR-INT-003) — nice-to-have or v2.1? — recommend v2.1
5. **Mobile-friendly form layouts** — desktop-first, but should responsive break at 768px work? — recommend yes for view-only, no for authoring

► 13 · Approvals

| ROLE               | NAME | STATUS  | DATE |
|--------------------|------|---------|------|
| Product owner      | TBD  | Pending | —    |
| Engineering lead   | TBD  | Pending | —    |
| Data platform lead | TBD  | Pending | —    |
| Security review    | TBD  | Pending | —    |

► Appendix A · References

- **v1.0 codebase:** `tools/data-factory-pipeline-studio.html` (7800+ LOC · 6 tabs · localStorage drafts)
- **v1.0 architecture slide:** `docs/data-factory-pipeline-architecture.html`
- **v1.0 CHANGELOG:** `CHANGELOG.md`
- **Agent prompts** (the IP): `agents/*.md` (6 files)
- **Skill catalogue:** `.github/skills/**/SKILL.md` (19 files)
- **Rule catalog:** `.github/rules/categories/*.yaml` (38 rules across 10 categories)
- **Path contract:** `.github/copilot-instructions.md` § 3
- **v2.0 architecture diagram:** `docs/v2-architecture.html` (companion to this document)

► Appendix B · Document control

| VERSION | DATE       | AUTHOR                                | CHANGE    |
|---------|------------|---------------------------------------|-----------|
| 0.1     | 2026-05-25 | Initial draft from v1.0 retrospective | First cut |